计算机应用
Journal of Computer Applications
ISSN 1001-9081,CN 51-1307/TP

《计算机应用》网络首发论文

题目：　基于开放领域 LLM 及改进 RAG 的企业智能问答系统方案
作者：　刘杰
收稿日期：　2024-12-10
网络首发日期：　2026-02-05
引用格式：　刘杰. 基于开放领域 LLM 及改进 RAG 的企业智能问答系统方案[J/OL]. 计算机应用. https://link.cnki.net/urlid/51.1307.TP.20260205.1129.002

网络首发：在编辑部工作流程中，稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定，且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式（包括网络呈现版式）排版后的稿件，可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定；学术研究成果具有创新性、科学性和先进性，符合编辑部对刊文的录用要求，不存在学术不端行为及其他侵权行为；稿件内容应基本符合国家有关书刊编辑、出版的技术标准，正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性，录用定稿一经发布，不得修改论文题目、作者、机构名称和学术内容，只可基于编辑规范进行少量文字的修改。

出版确认：纸质期刊编辑部通过与《中国学术期刊（光盘版）》电子杂志社有限公司签约，在《中国学术期刊（网络版）》出版传播平台上创办与纸质期刊内容一致的网络版，以单篇或整期出版形式，在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊（网络版）》是国家新闻出版广电总局批准的网络连续型出版物（ISSN 2096-4188，CN 11-6037/Z），所以签约期刊的网络版上网络首发论文视为正式出版。

网络首发时间：2026-02-05 15:29:50　网络首发地址：https://link.cnki.net/urlid/51.1307.TP.20260205.1129.002

文章编号：1001-9081(2026)S1-0001-10　DOI:10.11772/j.issn.1001-9081.2024121739

基于开放领域LLM及改进RAG的企业智能问答系统方案

刘　杰*

（中移（苏州）软件技术有限公司 平台产品部，江苏 苏州 215163）

（*通信作者电子邮箱 liujie@cmss.chinamobile.com）

摘　要：针对开放领域大语言模型（LLM）及企业业务数据微调（FT）作用于企业智能问答系统时存在的“幻觉”、知识更新效率低和成本高的问题，提出基于开放领LLM及改进模块化检索增强生成（RAG）的企业智能问答系统方案。该方案通过多次前K个计算及阈值限制和知识增量更新基本解决上述问题。首先，计算用户查询与信息源系统的相似性，取前K个系统并且剔除相似性小于阈值的系统；其次，计算下一个生成单元（token）的概率，若小于阈值，则重新生成，从而为每个信息源系统生成前K个最近似结果；最后，融合信息源系统向量查询和图查询的结果。实验结果表明，所提方案相较于FT方案正确率提升了15个百分点，单位知识库更新耗时减少了96%，服务器成本减少了85%，可见，所提方案提升正确率的同时实现了企业降本增效。

关键词：开放领域大语言模型；微调；改进模块化检索增强生成；企业智能问答系统；企业降本增效

中图分类号：TP399　　**文献标志码：**A

Solution of enterprise intelligent question answering system based on open-domain LLMand improved modular RAG

LIU Jie*

（*Department of Platform Products*，　*China Mobile*　（*Suzhou*）　*Software Technology Company Limited*，　*Suzhou Jiangsu* 215163，　*China*）

Abstract: To address the challenges of “hallucinations”, inefficient knowledge updates, and high costs when applying open-domain Large Language Model (LLM) and Fine-Tuning (FT) with enterprise business data to enterprise intelligent question answering systems, the solution of enterprise intelligent question answering system based on open-domain LLM and improved modular Retrieval-Augmented Generation (RAG) was proposed. In this solution, the aforementioned issues were addressed basically through multiple top-K computations, threshold limitations, and incremental knowledge updates. Firstly, the similarity between user queries and information source systems was calculated, and the top-K systems were selected while those with similarity below the threshold were eliminated. Secondly, the probability of the next generated token was computed, and if it was below the threshold, the regeneration would be performed until the top-K most similar results were generated for each information source system. Finally, the results of vector queries and graph queries of the information source systems were fused. Experimental results show that compared to the FT solution, the proposed solution has the accuracy increased by 15 percentage points, the time required for updating the knowledge base reduced by 96%, and the server cost decreased by 85%. It can be seen that while enhancing accuracy, the proposed solution achieves cost reduction and efficiency improvement for enterprises.

Key words: open-domain Large Language Model (LLM); Fine-Tuning (FT); improved modular Retrieval-Augmented Generation (RAG); enterprise intelligent question answering system; enterprise cost reduction and efficiency improvement

0　引言

智能问答系统是一种高级的信息检索系统，建立在人们对快速、准确地获取信息的需求基础上。智能问答系统能够理解用户意图，精准定位用户需要的信息。智能问答系统根据应用领域的不同，可以分为特定领域和开放领域。特定领域与开放领域最大的区别在于：特定领域是面向不同领域知识的，可以利用大量与领域相关的数据，相比开放领域更有实际应用价值[1]，企业智能问答则属于特定领域的范畴。

随着ChatGPT的横空出世，大语言模型（Large Language Model，LLM）受到了广泛的关注，并已成为目前人工智能领域最前沿的技术。LLM是使用大量文本数据训练的深度学习模型，可以较好地理解自然语言和生成自然、连贯、准确的文本。随着LLM的快速发展，各种LLM层出不穷，例如国外谷歌Gemini，Facebook Meta，微软MAI-1等，国内百度“文心一言”，阿里巴巴“通义千问”、华为“盘古”、腾讯“混元”、科大讯飞“星火”、京东“言犀产业”、商汤“日日新”等。

上述LLM均属于开放领域LLM，若将开放领域LLM直接用于智能问答系统，因缺少企业业务数据会产生“幻觉”问题，回答不准确，业内常见的做法是微调（Fine-Tuning，FT）开放领域LLM，满足特定领域的智能问题诉求，但它存在以下问题：1）可以降低“幻觉问题”，但面对不熟悉输入仍可能产生；2）存储静态知识，每次更新知识库都需要重新训练，无法做到增量更新；3）知识更新成本高，单台训练服务器GPU基本是基本是CPU服务器的6倍左右，投入产出比低。针对上述问题，本文提出基于开放领域LLM及改进模块化检索增强生成（Retrieval-Augmented Generation，RAG）方案，通过提升正确率来缓解“幻觉”问题，通过增量更新提升更新效率，减少系统建设成本。

1　相关工作

FT和RAG是2种用于提升自然语言处理（Natural Language Processing，NLP）模型性能的技术。这2种技术在不同的应用场景中发挥着重要作用，它们的本质都是为了提升效率。FT是将一个预训练好的模型应用于特定任务的过程。这个过程涉及使用针对特定任务标注的数据集对模型进行再训练，以使模型能更好地完成该任务，降低训练资源需求，提升训练效率；RAG是一种结合了信息检索和语言生成的方法，旨在解决传统生成式模型存在的问题，如幻觉（即生成与事实不符的信息），提升查询效率。本文旨在将2种技术运用于企业智能问答这个特定领域，对比它们的优劣势。

1.1　RAG和FT对比

表1为RAG和FT的对比结果，可以看出相较于FT，RAG更能胜任知识密集型任务，可以适应频繁的知识更新而无需重新训练。

收稿日期：2024-12-10；**修回日期：**2025-03-07；**录用日期：**2025-03-13。　**基金项目：**国家自然科学基金资助项目（U22B2063）。

作者简介：刘杰（1988—），男，江苏泰兴人，高级工程师，硕士，主要研究方向：人工智能、大数据。

表1 RAG和FT比较

技术	知识更新	外部知识	数据处理	模型风格	可解释性	计算资源	延迟和实时要求	减少幻觉	道德和隐私问题
RAG	直接更新检索知识库,保持信息最新,无需频繁重新训练,适合动态数据环境	擅长利用外部资源,适合文档或结构化/非结构化数据库	对数据加工和处理要求低	主要关注信息检索,整合外部知识,难完全定制行为或写作风格	可追溯到特定数据源答案,可解释性和可溯源性高	需要高效检索策略和大型数据库相关技术,保持外部数据源集成及数据更新	数据检索可能导致更高延迟	基于检索证据,不太容易产生幻觉	问题来源于外部数据库检索的文本
FT	存储静态数据,知识与数据更新需重新训练	可FT以对齐外部知识,但对频繁更改数据源不太实用	依赖高质量数据集,对有限数据集可能效果不显著	可根据特定语气或术语调整行为、写作风格或特定领域知识	像黑匣子,可解释性相对较低	需准备整理高质量训练数据集、定义微调目标及相应计算资源	无需检索即可响应,延迟较低	特定领域训练数据可减少幻觉,但面对不熟悉输入仍可能产生	问题因训练数据存在敏感内容

1.2 FT

FT是利用大型预训练语言模型执行下游任务的既定方式。它修改了所有语言模型参数,因此需要为每个任务存储完整副本,在存在许多下游任务的情况下,FT参数效率低下:每个任务都需要一个新的完整模型。为了解决这个问题,常见解决方式为减少需要训练的参数数量来降低FT过程中的计算成本,但它不能避免仍然需要训练资源,并且不太适用于知识密集型任务。常见的有Adapter、Prefix-Tuning和LoRA(Low-Rank Adaptation),旨在通过减少需要训练的参数数量降低FT过程中的计算成本。Adapter和Prefix-Tuning主要是通过向模型中添加新的组件(适配器或前缀向量)实现这一点,而LoRA则对现有模型的权重矩阵进行低秩分解。选择哪种方法取决于具体的应用场景、可用资源以及对模型性能的要求。例如,如果希望保持模型尺寸较小且计算成本较低,Adapter或Prefix-Tuning可能是更好的选择;而对于那些需要更高精度的任务,LoRA可能会提供更好的性能。

1.2.1 Adapter

Adapter方法是在预训练模型的每一层或部分层中插入小型的神经网络模块(即适配器),这些适配器可以学习特定任务的知识,而原始模型的参数保持不变。例如Bottleneck Adapter[2]、Residual Adapter[3]等,优点为:1)训练参数少,因此计算成本较低;2)可以很容易地应用于多任务学习或多模态任务。缺点为:1)需要为每个任务添加额外的参数,可能会增加模型的大小;2)对于某些复杂任务,仅通过调整适配器可能无法达到最佳效果。在实践中,Adapter已经被应用于各种NLP任务中,包括但不限于文本分类、命名实体识别、问答系统等,同时Hugging FaceTransformer库提供了对Adapter的支持,使研究人员和开发者能够方便地在项目中集成这一技术。随着研究的深入,Adapter技术也在不断发展和完善。例如,研究者们正在探索如何更好地初始化Adapter、优化其架构设计以及提高其性能表现。同时,对于如何有效地管理大量的Adapter实例(特别是在多任务环境下),也是一个重要的研究方向。

1.2.2 Prefix-Tuning

Prefix-Tuning[4]是在预训练模型前加入一个可学习的前缀向量序列,这个前缀向量会在特定任务上进行训练,而预训练模型本身的权重保持不变。Prefix-Tuning变种包括动态Prefix-Tuning[5]、多任务Prefix-Tuning[6]等。优点为:1)与Adapter类似,Prefix-Tuning也具有较低的计算开销;2)通过前缀向量的引入,可以在不改变原有模型结构的情况下实现任务定制化。缺点为:1)前缀向量的设计和长度选择可能需要一定的实验和调整;2)同样存在对于复杂任务可能表现不佳的问题。随着更多实验和应用案例的积累,预计会有更多的优化和改进。例如,研究者们正在探索如何更有效地设计前缀结构、选择最佳长度以及提高跨任务泛化能力。此外,Prefix-Tuning与其他轻量化FT方法(如Adapter)的结合也是一个值得关注的方向。

1.2.3 LoRA

LoRA是一种新的FT策略,它通过在预训练模型的权重矩阵中加入低秩矩阵进行微调。该方法允许使用较少的参数近似原模型的更新,从而减少了计算资源的需求。LoRA变体包括:分层LoRA[7],动态LoRA[8]等。优点为:1)相比直接FT整个模型,LoRA可以显著减少所需的训练时间和内存;2)通过控制低秩矩阵的秩,可以灵活地平衡模型性能和计算效率。缺点为:1)选择合适的秩值可能需要一些实验;2)在某些情况下,低秩近似可能不足以捕捉到所有必要的参数变化。随着更多实验和案例积累,未来可能会出现更多关于如何更有效地设计低秩矩阵、优化其架构配置以及提高跨任务泛化性能的研究。此外,LoRA与其他轻量化FT技术(如Adapter, Prefix-Tuning)的结合也是一个值得关注的方向。

1.3 RAG

RAG[9]是一种结合信息检索和自然语言生成的技术,旨在提高生成内容的正确率、相关性和时效性。RAG通过从外部知识库中检索相关信息增强生成过程,从而避免模型的幻觉问题,并增强对实时性要求较高问题的处理能力。

1.3.1 标准RAG

标准RAG方法通过从外部知识库中检索相关信息,然后结合这些信息与生成模型,生成最终的输出。检索阶段:使用信息检索技术(如TF-IDF、BM25、向量检索等)从外部知识库中检索与用户查询相关的文档或段落。生成阶段:将检索到的信息作为上下文输入到生成模型中,生成最终的输出。优点为:1)高正确率。通过检索外部知识库,可以提高生成内容的正确率。2)实时性。能够处理实时更新的数据,确保生成内容的时效性。3)减少幻觉。通过外部知识库的支持,减少生成模型的幻觉问题。但标准RAG主要依赖于简单的块相似性,这在面对复杂查询和内容多样时表现不佳,它的主要问题包括:

1)查询理解不深刻。查询与文档块之间的语义相似性并不总是匹配,单纯依赖相似性计算进行检索,无法深入探究查询与文档间的关系。

2)检索的冗余与噪声信息。直接将所有检索到的内容输入LLM,往往会干扰模型识别关键信息,增加生成错误和虚假信息的风险。

1.3.2 高级RAG(Advanced RAG)

为了解决标准RAG存在的问题,高级RAG[10]着重优化检索环节,旨在提高检索效率并更好地利用检索结果。典型策略包括查询重写(Query Rewrite),使检索目标更明确,提高检索准确度,以及对检索结果重新排序(Rerank),帮助LLM更好地识别和利用关键信息。

1.3.3 模块化RAG

尽管高级RAG在应用中取得了一定进展,但它的能力与实际需求之间仍有差距。具体地,RAG目前面临如下新挑战:

1)复杂数据源的集成。RAG系统不仅处理非结构化文本,还集成了半结构化数据(如表格)和结构化数据(如企业知识图谱)。访问这些异构数据源为系统提供了更加丰富的知识背景和更可靠的知识验证能力。

2)系统可解释性、可控性和可维护性的新增需求。随着系统复杂度的增加,维护和调试变得更加困难。当问题发生时,快速定位并优化特定组件显得尤为重要。

3)组件选择与优化。RAG系统中的神经网络越来越多,需根据任务和资源选择合适的组件。同时,额外组件虽增强了系统能力,但也带来了新的协同工作要求。确保各模型有效协作、提升整体性能成为关键。

4)工作流程编排和调度。各组件可能需按照特定顺序执

行,或在特定条件下并行处理,甚至由LLM根据不同输出进行判断。合理规划工作流程对于提升系统效率和实现预期效果至关重要

为应对RAG系统日益复杂的设计、管理和维护挑战,并满足不断增长的需求模块化RAG[11]应运而生。未来RAG可能有3个发展方向:

1)垂直优化。解决长下文长度限制、增强鲁棒性和安全性、探索RAG与FT的协同作用等问题。

2)多模态拓展。将RAG应用于图像、音频、视频等领域,并考虑多模态数据间的融合。

3)生态系统建设。扩大RAG的应用范围至推荐系统、信息抽取等多个领域,构建完整的应用生态。

2　系统设计和实现

已有学者开展了将RAG运用于智能问答系统的相关研究,通过捕捉用户问题的语义特征及结合状态识别观点和问答对话文本优化技术[12],生成更加贴近用户需求的应答文本,本文方案同样作用于智能问答系统,但创新点并不相同,方案主要通过查询与信息源系统的多次Top-K计算、阈值限制及融合多信息源系统的查询结果,提升查询正确率。

本节首先分层介绍了企业智能问答系统的系统架构,接着介绍了各层需要具备的能力,最后详细阐述了关键层的实现。

2.1　系统架构

系统架构如图1所示,从下到上分别是数据源层、索引层、存储层、改进模块化RAG层及查询层,其中数据源涵盖了多种类型,包括文本类、视频类、图片类,索引层为数据源创建索引,存储层中存储了数据源层及索引层相关信息,用于改进模块化RAG查询,查询层主要是面向用户的,用户输入查询条件,经过改进模块化RAG过滤后,反馈答案。

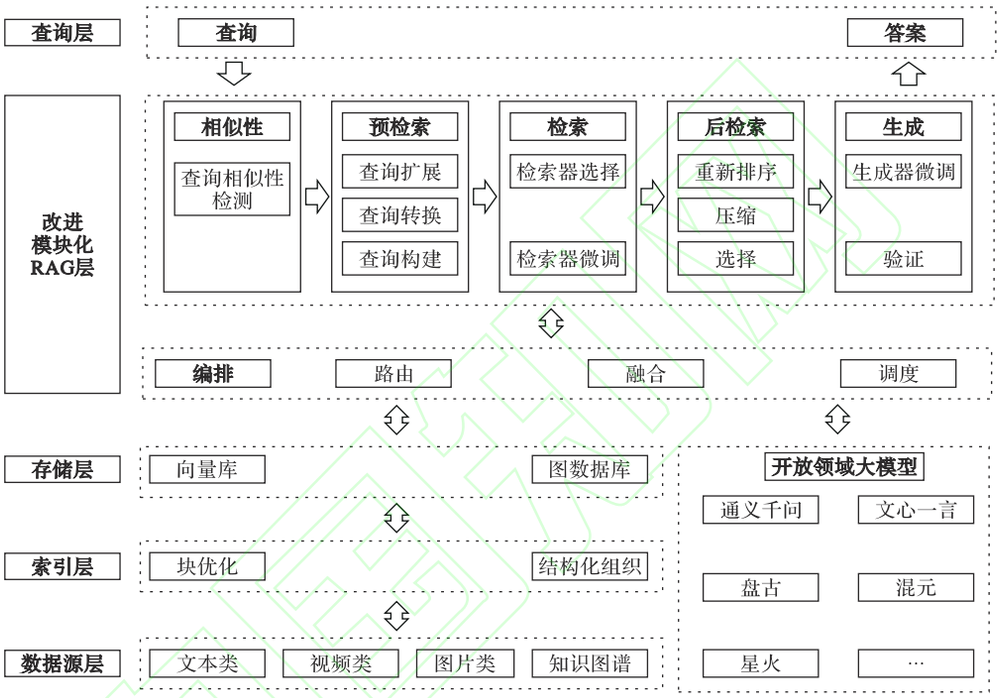


图1　系统架构

2.2　分层功能介绍

本节按照从下往上的方式逐一介绍各分层的功能,分别是数据源层、索引层、存储层、改进模块化RAG层、查询层。

2.2.1　数据源层

数据源层用于表示智能问答系统可查询的数据源,通常分为3类:结构化数据、半结构化数据和非结构化数据。结构化数据例如企业知识图谱,它属于结构化语义知识库;半结构化典型的例如XML、HTML;非结构化数据例如视频类、图片类。文本类常见的有txt、pdf和Word,其中pdf和Word中除了有文本信息,大概率会存在表格或者图片信息,视频类常见的有mp4等,这些数据源对于回答企业产品、项目、流程等问题有重要的参考意义,但常见的开放LLM因缺乏对这些数据源的训练,因此无法胜任企业问答这个特殊领域的问答问题。企业知识图谱是业务数据实体关系库,通常存储在图数据库中,其他数据源一般分散存储在各个系统(一般是内网系统)中,常见的比如产品管理系统、项目信息管理系统、OA系统,需要将这些系统中的数据通过爬虫或者接口方式采集、处理之后存储到存储层。

2.2.2　存储层

存储层用于将从各个系统采集到的数据经过索引处理后存储下来,其中文本类的数据一般存储到向量库,企业知识图谱类的数据一般存储到图数据库。模块化RAG层在收到用户查询请求的时候,会检索存储层的数据,用于回答用户的问题。

2.2.3　索引层&改进模块化RAG层

标准模块化RAG分为六大子模块,分别是索引、预检索、检索、后检索、生成和编排。本文在标准的模块化RAG基础上新增了相似性检测模块,用于缩减查询范围,提升查询效率和正确率。

1)索引。

索引负责将文档分割为可管理的内容,这是模块化RAG构建的关键,面临内容表达不完整、块相似度搜索不准确、引用链模糊问题。内容表达不完整:重要信息在较长的上下文中因为分割不同而被丢失或忽略。块相似度搜索不准确:数据增加使得检索噪声变多,导致错误数据匹配增加。引用链模糊:检索到的块可能来自不同文档,尽管内容在语义上相似,但实际可能涉及不同主题。常见的解决措施有块优化和结构化组织。

块优化　块的大小以及重叠程度直接影响RAG系统的性能:较大的块提供更多上下文信息同时会引入更多噪声,增加处理时间和成本;较小的块则噪声较小,则可能无法完整传达上下文[13]。为了提升效果,使用带有重叠的滑动窗口可以改善语义过渡,但也有控制上下文尺寸困难和语义失准的缺点。元数据附加可以为块添加元数据,如页码、文件名等,帮助缩小检索范围,提高精度。Small-to-Big方法[14]将用于检索的小块和用于生成的大块分开,通过先检索小块再参考大块来提升整体检索效果。

结构化组织　常见的有分层索引[15]和知识图谱索引[16]。分层索引:通过节点的父子关系组织文档内容,每个节点存储数据摘要,帮助快速提取关键信息。构建结构化索引的方法主要包括:基于段落和句子分割的结构感知;基于PDF、HTML和Latex等内在结构的内容感知;基于语义识别和分割的语义感知。知识图谱索引:利用知识图谱明确概念与实体间的关系,降低错配

风险，并提升检索准确度。知识图谱还将信息检索转化为语言模型可理解的指令，提升检索的正确率，并能生成语境连贯的响应，从而提高RAG系统的整体效率。

2)相似性检测。

通过匹配查询和信息源系统的点积相似性，按相似度取Top-*K*，可以有效缩减查询的范围。

3)预检索。

基础RAG的一大问题是过于依赖用户的原始查询，导致检索效果不佳。具体问题包括：查询措辞不当和语言中的歧义性，这可能导致错误检索结果。常见的解决措施有查询扩展、查询转换和查询构建。

①查询扩展。系统可以生成多个不同查询来丰富语义，确保生成的回答更加相关。多重查询利用提示词工程扩展查询，子查询[17]则将复杂问题分解生成多个子问题，并可通过并行或顺序执行来提高检索效果。

②查询转换。依据转换后的查询而非用户的原始查询进行检索和生成，常见的有查询重写[18]、HyDE(Hypothetical Document Embedding)[19]和回退提示(Step-back Prompting)[20]。

查询重写 原始查询在现实场景中的检索常常存在不足。为解决此问题，可以提示大型语言模型进行重写，也可运用专门的较小模型来达成此目的。

HyDE 为弥合问题与答案之间的语义差距，在回应查询时构建假设文档(假设答案)，而非直接搜索查询。它侧重于从答案到答案的嵌入相似性，而非寻求针对问题或查询的嵌入相似性。此外，它还包含反向HyDE，为每个块生成假设查询，并专注于从查询到查询的检索。

回退提示 原始查询被抽象为一个高级概念问题(回退问题)。在RAG系统中，回退问题和原始查询均用于检索，且将它们的结果相结合来生成语言模型的答案。

③查询构建。除文本数据外，诸如表格和图形数据等越来越多的结构化数据正被融入RAG系统。为适配各类数据类型，重构用户的查询很有必要。这涉及将查询转化为其他查询语言以访问替代数据源，常见方法有Text-to-SQL或Text-to-Cypher等。结构化查询语言(如SQL、Cypher)常与语义信息和元数据结合，用于构建更复杂的查询。

4)检索。

检索过程在RAG系统中举足轻重，提升检索效果需考虑的3个主要方面包括检索效率、质量以及任务、数据和模型的对齐。

①检索器选择。

随着RAG技术的广泛应用，嵌入模型的发展势头正盛。除基于统计的传统模型和基于编码器结构的预训练模型外，在LLM上FT的嵌入模型也展现出强大的能力[21]。然而，它们往往参数更多，导致推理和检索效率欠佳。故而，依据不同任务场景选择合适的检索器至关重要。

稀疏检索器(Sparse Retriever) 用统计方法将查询和文档转化为稀疏向量。其优势在于处理大型数据集时效率高，只关注非零元素。但在捕捉复杂语义方面，可能不如密集向量有效。常见方法包括TF-IDF和BM25。

密集检索器(Dense Retriever) 使用预训练语言模型(Pre-trained Language Model，PLM)为查询和文档提供密集表示。尽管计算和存储成本较高，却能提供更复杂的语义表示。典型模型有BERT结构的PLM，如ColBERT，以及多任务FT模型，如GTE[22]。

混合检索器(Hybrid Retriever) 同时使用稀疏和密集检索器。两种嵌入技术相互补充，以提高检索效果。稀疏检索器能提供初始筛选结果；此外，稀疏模型增强了密集模型处理含罕见实体查询的零样本检索能力，从而增强系统的稳健性。

②检索器FT。

用在上下文可能与预训练语料库有差异的情况下，尤其是在医疗保健、法律等具有大量专有术语的高度专业化领域。虽然这种调整需要额外付出努力，但能大幅提高检索效率和领域对齐度。

监督微调(Supervised Fine-Tuning，SFT) 基于有标记的领域数据FT检索模型通常借助对比学习来实现。这涉及缩短正样本间的距离，同时拉大负样本间的距离。

语言模型监督检索器(Language Model Supervised Retriever，LSR) 与直接从数据集中构建FT数据集不同，LSR借助LM生成的结果作为监督信号，在RAG流程中对嵌入模型进行FT。

适配器(Adapter) 对大型检索器进行FT可能成本很高，尤其是在处理像gte-Qwen这类基于LLM的检索器时，通过融入一个适配器模块并进行FT可以缓解此情况。添加适配器的另一个好处是能够更好地与特定的下游任务相适配[23]。

5)后检索。

将所有检索到的块直接输入到LLM并非最佳选择，对块进行后处理有助于更好地利用上下文信息。主要的问题包括：

①中间内容丢失(Lost in the middle)。像人类一样，LLM往往只记得长文本的开头或结尾，而遗忘中间部分[24]。

②噪声/反事实块(Noise/anti-fact chunks)。检索到的有噪声或事实上相互矛盾的文档可能会影响最终的检索生成[25]。

③上下文窗口(Context Window)。尽管检索到了大量相关内容，但大型模型中上下文信息长度的限制导致无法涵盖所有这些内容。常见的解决措施有重新排序、压缩和选择。

a)重新排序。

在不改变其内容或长度的情况下对检索到的块进行重新排序，以提升更关键文档块的可见性。常见有基于规则和基于模型重新排序。

基于规则重新排序 依照某些规则计算指标来对块进行重新排序，常见的指标包括：多样性、相关性和最大边际相关性(Maximal Marginal Relevance，MRR)[26]。其思路是减少冗余并增加结果的多样性。MRR基于查询相关性和信息新颖性的组合标准为最终的关键短语列表选取短语。

基于模型重新排序 利用语言模型对文档块重新排序，通常依据块与查询之间的相关性。重新排序模型已成为RAG系统的重要组成部分，相关模型技术也在不断迭代升级。重新排序的范围还扩展到了多模态数据，比如表格和图像[27]。

b)压缩。

在RAG流程中，人们普遍认为，尽可能多地检索相关文档，并将它们串联起来形成冗长的检索提示，是有益的；但这种认知其实是个误区，过多的上下文信息可能会带来噪声，影响LLM捕捉关键信息的能力。为了解决这个问题，通常采用压缩和筛选检索内容的方法。LLMLingua[28]：利用经过对齐和训练的小语言模型，比如GPT-2 Small或LLaMA-7B，可以从提示中识别并剔除不重要的标记，将它们转化为一种对人类来说晦涩难懂，但对LLM却易于理解的形式。这种方法提供了一种直接而实用的提示压缩方式，无需对LLM进行额外训练，同时保持了语言的完整性和压缩比的平衡。

c)选择。

与压缩文档内容不同，选择是直接移除无关的文档块。常见的有上下文和LLM选择。

上下文选择 通过识别并剔除输入上下文中的冗余内容，可以精炼输入，从而提升语言模型的推理效率。在实际操作中，选择性上下文根据基础语言模型计算的自信息来评估词汇单元的信息量。通过保留自信息更高的内容，这种方法提供了一种更简洁、高效的文本表示方式，且不会影响它在多样化应用中的表现。然而，这种方法忽略了压缩内容与目标语言模型以及用于提示压缩的小语言模型之间的相互依赖性[29]。

LLM选择(LLM-Critique) 另一种直接而有效的方法，是在生成最终答案前，让LLM评估检索到的内容。这允许LLM通过LLM批评机制，过滤掉相关性不高的文档。例如，在Chatlaw[30]中，LLM被引导自我推荐相关法律条款，以评估其相关性。

6)生成。

利用LLM根据用户的查询和检索到的上下文信息生成答案。选择一个符合任务需求的合适模型，考虑FT需求、推理效率和隐私保护等因素。

①生成器FT。

除了直接使用LLM外，根据场景和数据特征进行有针对性

的FT,可以带来更好的效果。这也是使用本地部署LLM的一个显著优势。

指令微调(Instruct-Tuning)　当LLM在特定领域缺少数据时,可以通过微调为它提供额外的知识。通用的微调数据集可以作为第一步。微调的另一个好处是能够调整模型的输入和输出,例如,使LLM适应特定的数据格式,并根据指令以特定风格生成响应[31]。

强化学习(Reinforcement Learning)　通过强化学习来让LLM的输出与人类或检索器的偏好相契合,是一种颇具潜力的方法[32]。例如,对最终生成的答案进行手动标注,然后借助强化学习给予反馈。除了与人类的偏好达成一致,与FT模型和检索器的偏好相符合也是可行的。

双重微调(Dual Fine-tuing)　同时对生成器和检索器进行微调,以实现它们偏好的一致。一种典型的方式,如RA-DIT[33]、利用KL散度对齐检索器和生成器之间的评分函数。

②验证(Verification)。

尽管RAG提升了LLM生成答案的可靠性,但在众多场景中,它需要将幻觉出现的概率降至最低。因此,可以通过额外的验证模块来筛除不符合要求标准的响应。常见的验证方法包含基于知识库和基于模型。

基于知识库的验证　通过外部知识直接对大型语言模型生成的响应进行验证。通常,先从响应中提取特定的陈述或三元组。而后,从经过验证的知识库(如维基百科或特定的知识图谱)中检索相关证据。最后,将每个陈述与证据逐步比对,以判定该陈述是得到支持、被驳斥还是信息不足[34]。

基于模型的验证　运用小型语言模型来验证大型语言模型生成的响应[35]。给定输入问题、检索到的知识和生成的答案,训练一个小型语言模型来判别生成的答案是否正确反映了检索到的知识。此过程被设定为一道选择题,验证器需要判断答案是否反映了正确答案。如果生成的答案未正确反映检索的知识,可迭代地重新生成答案,直至验证器确认答案正确。

7)编排。

编排涉及管理RAG过程的控制模块。与传统固定流程的僵化方式不同,RAG如今在关键节点纳入决策,并依据先前的结果动态选择后续步骤。这种自适应和模块化的能力是模块化RAG的显著特征,使它有别于更为简单的朴素和高级RAG范式。它包括路由、调度和融合。

①路由。

面对不同的查询,RAG系统会路由到为不同场景定制的特定管道,这是为应对各种情况而设计的多功能RAG架构的基本特性。需要一个决策机制,依据来自模型的输入或补充的元数据,来确定将启用哪些模块。针对不同的提示或组件采用不同的路由。

②调度。

RAG系统在复杂性和适应性方面不断演进,能够借助复杂的调度模块来管理流程。调度模块在模块化RAG中发挥着关键作用,能识别需要外部数据检索的关键节点,评估响应的充分性,并决定是否需要进一步的调查。它通常用于涉及递归、迭代和自适应检索的场景,确保系统能就何时停止生成或启动新的检索循环做出明智决策。常见的有规则判断、LLM判断和知识驱动调度。

规则判断　后续步骤由一组既定规则支配。通常,系统通过评分机制评估生成答案的质量。继续或停止该过程的决策取决于这些分数是否超过某些预定阈值,这些阈值通常与单个token的置信水平有关,接受的条件是其中的所有令牌必须具有大于或等于阈值的相关概率。若不满足此条件,系统会重新生成新答案。

LLM判断　LLM独立决定后续的行动。有两种主要方式来实现这种能力:第一种方式利用LLM的上下文学习能力,并通过提示工程进行判断。这种方式的一个显著优点是无需对模型进行FT;然而,判断输出的格式取决于LLM对所提供指令的遵循情况。第二种方式涉及LLM生成特定token,通过FT启动有针对性的行动。这种技术源自Toolformer[36],已被集成于像Self-RAG[37]这样的框架。这使得对LLM的行动有了更直接的控制机制,增强了系统对对话上下文中特定触发因素的响应能力;不过,它需要生成大量符合要求的指令集来FT LLM。

知识驱动的调度策略(Knowledge-guided scheduling)　突破了基于规则的方法的局限,也不再完全依赖LLM控制流程,而是采用了一种更灵活的中间策略:知识引导的调度[38]。通过知识图谱的强大功能,引导检索和生成过程。从知识图谱中提取与问题相关的信息,构建一个由逻辑上相互关联的节点组成的推理链,每个节点都承载着解决问题的关键信息。基于这一推理链的节点信息,可以分别进行信息检索和内容生成。整合这种方法,不仅提升了问题解决的效率和精确度,也使得提供的解释更加清晰明了。

③融合。

随着RAG进程的发展,它已经超越了单一的线性流程,经常需要扩大检索范围或通过探索多条流水线来增加多样性。在扩展到各个分支之后,融合模块有效地整合了信息,确保了全面且连贯的响应。融合模块的作用不仅是合并答案,还要确保最终输出内容丰富,并且能够反映出问题的多维度特性。常见的有LLM融合、加权集成和互反排名融合。

LLM融合(LLM fusion)　将多分支的信息聚合起来最直接的方法之一是利用LLM的强大功能来分析和整合不同分支的信息。然而,这种方法在处理超出LLM上下文窗口限制的长答案时也面临挑战。为了解决这个问题,通常的做法是先对每个分支的答案进行总结,提取关键信息,然后再将它们输入到LLM中,以确保即使在长度受限的情况下也能保留最重要的内容。

加权集成(Weighted ensemble)　从多个分支生成的不同标记的加权值,以实现对最终输出的全面选择。权重是根据文档与输入查询之间的相似性得分来确定的。权重通过使用softmax函数来计算,确保了权重的归一化,并且总和为一。

互反排名融合(Reciprocal rank fusion)　将多个检索结果的排名合并成一个统一的列表[39]。它采用定制的加权平均方法来提高整体的预测性能和排名精度。优势在于动态权重分配,这种分配是由分支之间的相互作用所决定的。在模型或来源的异质性特别显著的场景中,RRF尤其有效,它可以显著提高预测的正确率。

2.2.4　查询层

查询层以对话框的形式实现人机交互,用户可以在对话框中提出相关问题即查询,系统在对话框中返回结果即答案。

2.3　关键层实现

本文选择的开放领域LLM为千问Qwen2.5-14B-Instruct-GGUF,依托该模型以及改进模块化RAG,完成企业智能问答系统的构建。相较于模块化RAG,本文新增相似性检测及融合向量查询和图查询结果作为生成输出。相似性检测即检测查询和信息源系统的相似性,取相似度较高的前K个,缩小了查询的范围;融合向量查询和图查询结果(token低于阈值,重新生成),上述措施提升了查询效率和精确度。下文首先介绍了用户查询的流程,随后重点介绍数据源层、索引层、存储层以及改进模块化RAG层实现。

2.3.1　用户查询流程

用户查询流程如图2所示,以用户查询和结果返回为例,阐明了改进模块化RAG各个子模块的具体方案以及各模块之间的交互。

2.3.2　数据源层数据索引持久化到存储层

由于企业各系统的数据大多数为PDF、Word、HTML,和JSON等文本类或者知识图谱,为了实现模块化RAG索引部分的优化,将块优化和结构化组织结合起来完成索引优化,块优化通过添加文本元数据信息到块元数据信息,结构化组织通过分层索引和知识图谱索引完成,将文本类的数据存入向量库,将知识图谱中实体、属性、关系、文本索引存入图数据库。具体持久化方案如图3所示。

1)企业知识图谱存储。

企业知识图谱存储各系统涉及的实体、关系、属性。作为产品研发公司,以产品管理和项目管理系统为中心,其他系统为辅

助，可关联产品及项目的人员、会议、合同、预算、报账情况，用该信息作为生成结果的补充会更丰富和准确。围绕产品、项目常见系统有：产品管理、项目管理、会议管理、通讯录、合同管理、预算管理、报账管理和外协管理。上述系统涉及实体、关系、属性如图4所示。后续如有系统新增，将新系统实体、关系、属性融入已有图谱。

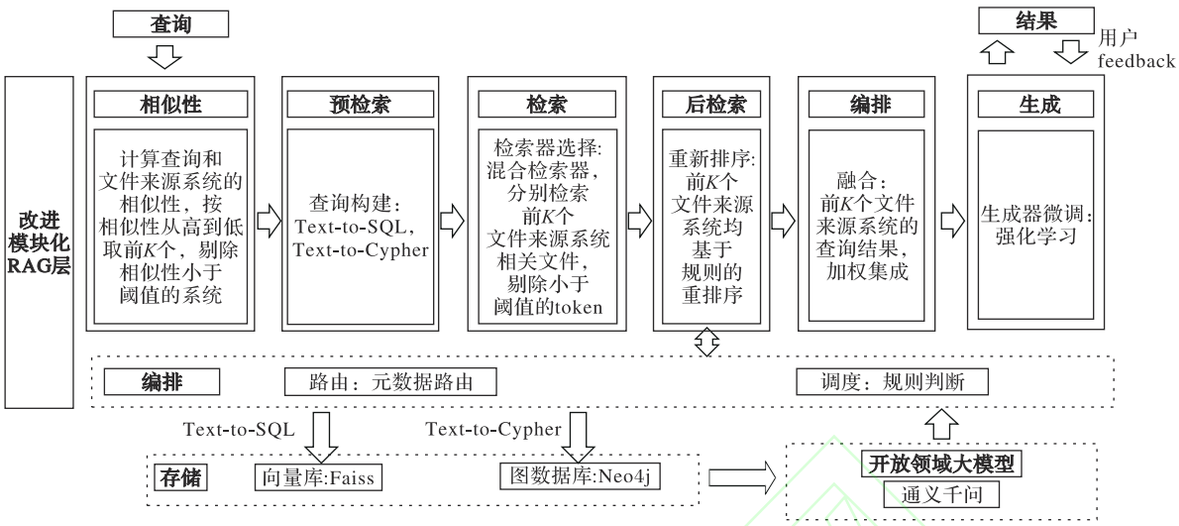


图2 用户查询流程

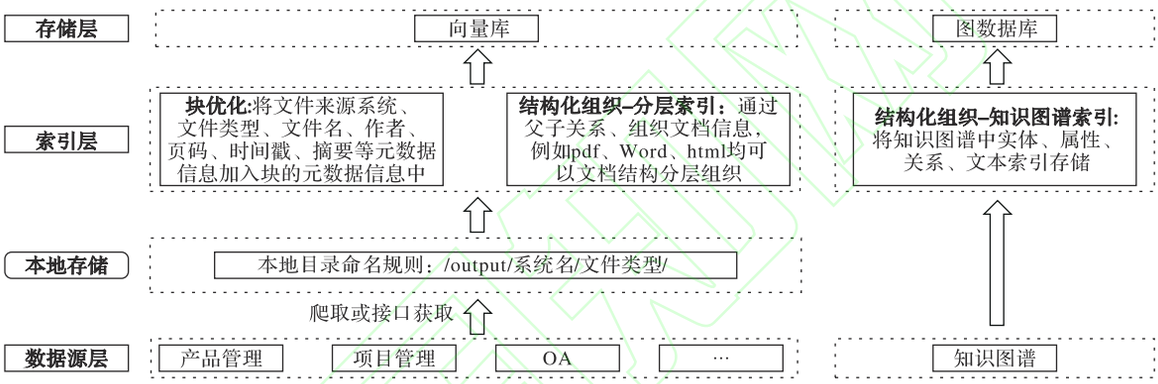


图3 数据源持久化到存储层

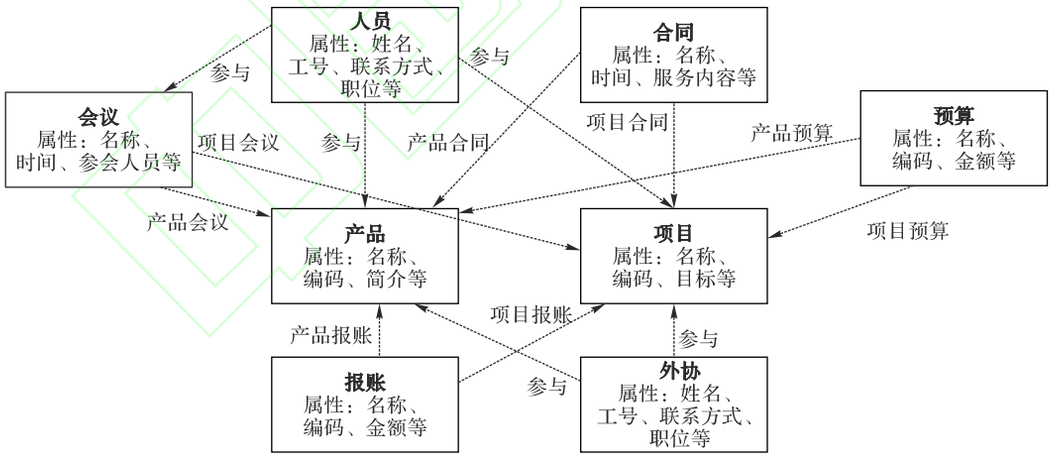


图4 企业知识图谱构建

2)其他数据源存储。

通过爬虫爬取各系统的数据，并按照/output/系统名/文件类型分级存储，将爬取的数据通过块优化及分层索引的方式存储到向量库。其中块优化包括：将文件来源系统、文件类型、文件名、作者、页码、时间戳、摘要等元数据信息加入块的元数据信息中。分层索引：通过父子关系，组织文档信息，例如pdf、Word和html均可以文档结构分层组织。

2.3.3 改进模块化RAG层

模块化RAG的算法及其实现，为了便于算法描述，定义符号如表2所示。

算法1 模块化RAG。

输入 原始查询q、文档集合D、语言模型LLM、相似性函数$Sim(q,S)$、检索器R，前K个结果，合并模块M_{merge}；

输出 最终输出$\hat{y}$。

1) 初始化：
2) $S_k = \text{Top-}K(Sim(q,S))$
   //根据原始查询，计算查询最可能和哪K个数据源系统相关
3) $O = \varnothing$ //初始化一个空集，存储查询结果
4) For all $s_i \in S_k$ do
5) If $P(s_i) < \eta$ then //剔除小于阈值的系统
6) Continue
7) End if
8) $q' \leftarrow \text{QueryTransform}(q)$ //将查询转为SQL和Cypher
9) $D' \leftarrow R_{dr}(q', f_r(R_{sr}(q', D)))$
   //根据信息源系统信息检索向量库，图数据库

10）　For all $d'_{ij} \in D'_i$ do
11）　$y_{ij} \leftarrow \mathrm{LLM}(s_i, d_{ij})$　　//为每个数据源系统生成结果
12）　If $P\left(y_{ij} \middle| s_i, d_{ij}\right) \geqslant \tau$ then
13）　$O_i \leftarrow O_i \cup \{y_{ij}\}$
//将token的概率大于等于阈值τ添加至结果集合
14）　Else then
15）　While $P\left(y_{ij} \middle| s_i, d_{ij}\right) < \tau$ do
16）　$y_{ij} \leftarrow \mathrm{LLM}(s_i, d_{ij})$
//token的概率小于阈值τ，则重新生成
17）　If $P\left(y_{ij} \middle| s_i, d_{ij}\right) \geqslant \tau$ then
18）　$O_i \leftarrow O_i \cup \{y_{ij}\}$
//将重新生成token概率大于等于阈值τ添加至结果集合
19）　Break
20）　End if
21）　End while
22）　End if
23）　$O_i \leftarrow \mathrm{TopK}(f_{\mathrm{rerank}}(O_i))$
//基于相似性重排序每个系统的生成结果，取前K个
24）　End for
25）　$\hat{y} \leftarrow M_{\mathrm{merge}}(O_i)$　　//合并前K个数据源系统查询结果
26）　End for
27）　return $\hat{y}$

表2　算法符号说明

变量符号	含义
q	原始查询
y	LLM生成的输出
D	由多个块组成的文档检索库
S	数据源来自哪些系统集合，S_k代表前k个系统
Sim	相似性计算函数，例如$Sim(q,S)$根据q和S，计算相似性
$\mathrm{R}(q,D)$	检索函数，根据查询q检索文档库D中的相似块，***Rsr***代表稀疏检索，***Rdr***代表密集检索
Top-K	函数，从高到低，取前K个结果
f_r	RAG流程中路由功能
f_{rerank}	RAG流程中重排序，从高到低
M	模块化RAG中的模块
F	经过路由映射后的RAG流程

1）相似性检测。

检测查询(q)和信息源系统集合(S)的点积相似性，按高到低排序取前K个。计算方式如式(1)所示：

$$Sim(q, s_i)_{s_i \in S} = e_q \cdot e_{s_i} \tag{1}$$

2）预检索。

查询构建：Text-to-SQL，Text-to-Cypher中会将相似性检测到的信息源系统作为查询条件传入，前者将查询转为SQL，从向量库Faiss中查询结果，后者将查询转为Cypher，从图数据库Neo4j中查询结果。

3）检索。

检索器选择：稀疏检索初步筛选，密集检索生成文档，按相似性检测获取信息源系统分类检索结果。

4）后检索。

重新排序：基于相似度的从高到低，重排序基于信息源系统检索到的文档，取前K个，提升关键文档的优先级。

5）编排。

路由和融功能中涉及计算方式如式(2)~(5)所示，路由：元数据路由，定义基于信息源系统的RAG流程F_i，及流程预定义关键词K_i，从查询中提前关键词K'_i，计算关键词匹配得分：

$$score_{\mathrm{key}}(q_i, F_j) = \frac{1}{\left|K'_j\right|}\left|K_i \cap K'_j\right| \tag{2}$$

再根据关键词评分最高的关键词选取流程。

$$F_i(q) = \underset{F_j \in F}{\operatorname{argmax}}\ score\ (q, F_j) \tag{3}$$

融合：加权集成，计算不同信息源系统生成的结果概率、文档和查询系统相似性加权，选择加权值最高的输出作为结果，计算公式：

$$p\left(y \middle| s, D'\right) = \sum_{d \in D'} p\left(y \middle| d, s\right) \cdot \lambda(d, s) \tag{4}$$

其中：$p\left(y \middle| d, s\right)$代表不同信息源系统生成的结果概率，$\lambda(d, s)$由文档和查询系统$s$之间的相似度决定，e代表自然常数，通过softmax函数计算，归一化权重，$p\left(y \middle| s, D'\right)$为最终加权计算结果。

$$\lambda(d, s) = \mathrm{e}^{sim(d,s)} \Big/ \sum_{d \in D'} \mathrm{e}^{sim(d,s)} \tag{5}$$

6）生成。

生成器微调：强化学习，基于用户反馈的强化学习，通过收集用户对于查询结果评分反馈，基于这个反馈微调后续生成结果，将LLM输出结果和人类偏好对齐。

2.3.4　FT

基于通过爬虫存储的内部系统数据在/output/系统名)，将该目录下各类数据源转为Text格式后，用于FT。为方便描述算法，定义符号如表3所示。

表3　FT算法符号说明

变量符号	含义
model	模型
save_pretrained	保存预训练模型函数
load_dataset	模型加载训练数据函数
preprocess_function	预处理功能函数，主要用于文本分词，生成token
training_args	训练参数
trainer	训练器函数
tokenized_datasets	数据分词后的token数据集
Adapter	生成适配器函数
adapter_config	适配器参数
add_adapter	添加适配器函数
save_adapter	保存适配器函数
prefix_tuning	生成前缀模型函数
prefix_length	前缀长度
prefix_embedding	前缀向量
prefix_tokens	前缀token张量
LoRA	生成LoRA模型函数
lora_config	LoRA参数

1）Adapter

训练所有系统数据，为每个系统生成对应Adapter。

算法2　Adapter。

输入　系统数据；

输出　该系统对应的Adapter。

1）初始化：
2）for all $s_i \in S$ do　　//遍历所有系统数据
3）　*tokenized_datasets*=preprocess_function(load_dataset(s_i))
//加载系统数据，生成token
4）　*model*.add_adapter(Adapter(*adapter_config*))
//初始化Adapter，并添加至模型
5）　trainer(model,*training_args*,*tokenized_datasets*)
//将模型、训练器参数、训练token传给训练器，完成训练
6）　*model*.save_adapter(Adapter)　　//保存适配器
7）end for

2）prefix-tuning。

训练所有系统数据，保存每个系统对应的prefix模型。

算法3　prefix-tuning。

输入　系统数据；

输出　该系统对应的前缀模型。

1）初始化：
2）for all $s_i \in S$ do　　//遍历所有系统数据
3）　tokenized_datasets=preprocess_function(load_dataset(s_i))
//加载系统数据，生成token
4）　*model*=prefix_tuning(*model*,*prefix_embedding*,*prefix_tokens*)
//将前缀向量和前缀token传入，初始化前缀模型
5）　trainer(*model*, *training*_ arg s, *tokenized_datasets*)
//将模型、训练器参数、训练token传给训练器，
//训练过程中拼接模型的中前缀和系统数据token

6) *model*.save_pretrained() //保存训练后的前缀模型
7) end for

3)LoRA。

训练所有系统数据,保存每个系统对应的LoRA模型。

算法4 LoRA。

输入 系统数据;

输出 该系统对应的LoRA模型。

1) 初始化:
2) for all $s_i \in S$ do //遍历所有系统数据
3) *tokenized_datasets*=preprocess_function(load_dataset(s_i)) //加载系统数据,生成token
4) *model*=LoRA(*model*, *lora_config*)
5) trainer(*model*, *training_*arg*s*, *tokenized_datasets*) //将模型、训练器参数、训练token传给训练器
6) *model*. save_pretrained() //保存训练后的LoRA模型
7) end for

3 实验与结果分析

通过智能问答系统的正确率、效率、成本维度来评价改进模块RAG和FT的优劣势,可以看出正确率方面前者优于后者,效率方面响应时间前者略低于后者,但其知识更新时间、成本远低于后者。

3.1 数据集设置

数据来源公司内部系统,通过爬虫或者接口抓取OA、产品管理、项目管理、制度流程、会议管理、通讯录、合同管理、预算管理、报账管理、外协管理共10个公司系统文本类数据(包括pdf,docx,xlsx,json,html以及各个系统的FAQ)、企业知识图谱。爬虫获取的数据按/系统名称/文件类型/分级存储,知识图谱数据存储了各系统涉及的实体、属性及关系,构建方式详见2.3.2节。

3.2 实验设置

3.2.1 实验环境

实验使用到的软件、硬件信息如表4所示。

3.2.2 实验参数

微调和改进模块化RAG使用的参数如表5所示。

表4 软硬件环境

类型	名称	版本或数量、配置
软件	Python	3.12.6
	Pytorch	2.3.0
	操作系统	欧拉21.10
LLM	千问	Qwen2.5-14B-Instruct-GGUF
硬件	GPU服务器	3台,配置:2*Intel6248R(3.0 GHz,24cores)/24*32G/2*480GSSD/4*1.92T NVMe/[1*Raid0&1]/2*10GE/4*25GE/8*NVIDIAA100/
	CPU服务器	3台,配置:2*Intel 6248R(3.0 GHz,24cores)/24*32G/2*480G SSD/8*7.68T NVMe/[1*Raid0&1]/2*1GE/4*25GE/

表5 实验参数

类别	参数	描述	默认值
阈值	K	按相似性从高到低排序,取前K个	8
	η	低于该值的系统剔除	0.7
	τ	低于该值的token剔除,重新生成。	0.6
训练参数	*num_train_epochs*	训练过程中遍历整个训练数据集的次数	15
	per_device_eval_batch_size	训练时每个设备上的批处理大小	8
	per_device_train_batch_size	评估时每个设备上批处理大小	16
	weight_decay	惩罚项系数,用于优化过程中对模型权重应用L2正则化	0.01
	evaluation_strategy	评估策略	epoch
	learning_rate	学习率	5e-5
	warmup_steps	预热步数,用于训练开始阶段	1 000
Adapter	*mh_adapter*	是否在多头注意力机制层之后添加适配器	True
	output_adapter	是否在输出层之后添加适配器	True
	reduction_factor	适配器隐藏层大小相对于主模型隐藏层的比例	16
prefix-tuning	*prefix_length*	前缀的长度	150
LoRA	*task_type*	任务类型	QUESTION_ANSWERING
	inference_mode	是否推理,否为训练	False
	r	低秩矩阵的秩	8
	lora_alpha	LoRA的缩放因子	16
	lora_dropout	LoRA层的dropout率	0.05

3.3 消融实验

通过实验对比改进模块RAG和FT(Adapter,Prefix-Tuning,LoRA)在智能问答系统的正确率、效率、成本,前者综合优于后者。统计编号为1~10的10个系统,计算每个系统15个问题求平均正确率,发现其平均正确率和稳定性优于FT,平均正确率:较FT提升15个百分点,FT后约80%,改进模块RAG约95%,稳定性:改进模块RAG方差约1.7,FT的方差约4.4。效率考虑查询响应时间和更新更新时间,虽然平均响应时间相较于FT多6 s,FT约9 s,改进模块化RAG约15 s,两者均属于可接受范围;但是平均知识更新耗时减少96%,FT约20 000 s,改进模块RAG约800 s。服务器成本减少85%,FT为195万(3台GPU),改进模块RAG为30万(3台CPU)。综上改进模块化RAG优于FT,可有效缓解"幻觉问题"、提升知识库更新效率,降低知识更新成本。

如图5所示,改进模块RAG的正确率、知识更新时间、较FT有较大的改善,特别是知识更新时间。正确率高的原因有2个方面:1)新增查询和对应系统的相似度检测,只有高于阈值的系统才会加入查询;2)对于生成的token计算概率,低于阈值的会剔除,因此保证了正确率;但也因为这些额外的计算,它的响应时间劣于FT。同样更新1 GB知识数据,改进模块RAG知识更新时间效率高的原因是它支持增量的更新,新增的数据实时更新到存储层即可,可以做到分钟级,但是FT对于新增的知识需要重新训练,一般需要数小时甚至数天。粗略估算改进模块化RAG数据索引处理平均速度为1.3 MB/s,它的处理时间为787 s;FT处理时间依赖于GPU算力和LLM参数,耗时计算如式(6)所示:

$$\frac{8*\text{模型参数}*\text{token数量}}{\text{GPU数量}*\text{GPU算力}*\text{GPU使用率}} \tag{6}$$

每个参数、token结合前向、后向、激活重计算,共计需要8次浮点计算,模型参数为14 B,token数量1 B,GPU卡24块(3台,每台8卡),单卡峰值算力312T FLOPs,使用率80%,代入可得19 603 s。

通过本文方案和知识图谱分组消融实验,来验证方案中各组成对于正确率、响应时间的影响。实验表明系统相似性检测提升正确率约10个百分点。阈值token限制对响应时间影响大,占总时间约40%,知识图谱作为数据源提升了正确率约4个百分点,但未明显增加响应时间。

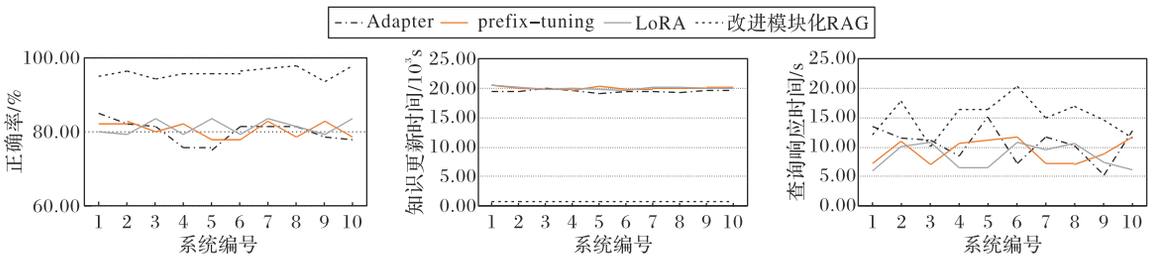
1)本文方案消融组。

A组:本文提出的方案。

B组:本文方案剔除查询与系统相似性检测。

C组:在B的基础上剔除生成token阈值限制。

对比结果如表6所示。A组和B组数据对比表明:查询和系统相似性检测对于正确率的影响大,其提升正确率10.95个百分点,但响应时间相比B组增加0.61 s。正确性提升原因是通过查询与系统相似性检测,增加了查询和系统的相关性,从而提升了正确率;响应时间增加不明显是因为查询和系统相似性检测不涉及到数据检索,因此耗时较少。B组和C组对比表明重新生成不满足阈值的token提升正确率2.97个百分点,但重新生成token耗时5.87 s,说明重新生成token对于查询响应时间影响较大。



(a) 正确率　(b) 知识更新时间　(c) 查询响应时间

图5　FT和改进模块RAG性能对比

表6　本文方案消融对比

分组	正确率/%	响应时间/s
A	95.66	16.25
B	84.71	15.64
C	81.74	9.77

2)知识图谱消融组。

D组:改进RAG,使用系统数据作为生成数据源。

E组:FT-Adapter,使用系统数据作为生成数据源。

F组:FT-Prefix-Tuning,使用系统数据作为生成数据源。

G组:FT-LoRA,使用系统数据作为生成数据源。

H组:E组基础新增知识图谱作为生成数据源。

I组:F组基础新增知识图谱作为生成数据源。

J组:G组基础新增知识图谱作为生成数据源。

对比结果如表7所示,A组和D组对比,可以看出增加知识图谱提升正确率4.03个百分点,减少知识图谱响应时间减少了1.66 s。正确率提升原因是知识图谱可以丰富生成内容,响应时间减少不明显原因是知识图谱属于结构化知识库,其查询本身占用响应时间就不大。同理对E组和H组、F组和I组、G组和J组进行对比,结论和原因一致。

表7　知识图谱消融对比

分组	正确率/%	响应时间/s
A	95.66	16.25
D	91.63	14.58
E	76.72	8.19
F	77.42	8.31
G	79.58	7.59
H	79.84	10.78
I	81.11	9.46
J	81.36	8.84

4　结语

本文提出了基于开放领域LLM+改进模块化检索增强生成(RAG)方案,来满足企业智能问答这个特定领域场景,通过系统相似性检测、token相似性检测,剔除不满足阈值的系统、token,可显著提升正确率和知识更新效率,降低知识更新成本,同时通过消融实验证明了方案中各组成部分对于正确率查询响应的影响,但仍然存在一些改进方向:

1)更丰富的知识类型。

目前系统主要还是处理文本类数据,后续可以增加图片、视频类数据的解析能力。

2)生成器微调改进。

目前是基于用户反馈的评分用于对齐LLM输出结果和人类偏好,相对而言较简单,后续可以考虑训练用户反馈和评分,使得生成的结果更符合用户的偏好。

3)敏感数据安全性。

企业中业务敏感数据需要分类、分级管理,并且具备权限管控能力,避免出现不应该获取敏感业务数据的用户,通过智能问答系统获取相关数据。

参考文献(References)

[1] 王东升,王卫民,王石,等. 面向限定领域问答系统的自然语言理解方法综述[J]. 计算机科学, 2017, 44(8): 1-8, 41.

[2] YANG H, WANG M, XIE N, et al. Efficient transfer learning for quality estimation with bottleneck adapter layer[C]// Proceedings of the 22nd Annual Conference of the European Association for Machine Translation. [S. l.]: European Association for Machine Translation, 2020: 29-34.

[3] TOMANEK K, ZAYATS V, PADFIELD D, et al. Residual adapters for parameter-efficient ASR adaptation to atypical and accented speech[C]// Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. Stroudsburg: ACL, 2021: 6751-6760.

[4] LI X L, LIANG P. Prefix-tuning: optimizing continuous prompts for generation [C]// Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers). Stroudsburg: ACL, 2021: 4582-4597.

[5] LIU X, HUANG H, SHI G, et al. Dynamic prefix-tuning for generative template-based event extraction [C]// Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Stroudsburg: ACL, 2022: 5216-5228.

[6] ZHANG Z, WANG S, XU Y, et al. Task compass: scaling multi-task pre-training with task prefix [C]// Findings of the Association for Computational Linguistics: EMNLP 2022. Stroudsburg: ACL, 2022: 5671-5685.

[7] GUO Z, ZHANG Y, ZHANG Z, et al. FedHLT: efficient federated low-rank adaption with hierarchical language tree for multilingual modeling [C]// Companion Proceedings of the ACM Web Conference 2024. New York: ACM, 2024: 1558-1567.

[8] HUANG C, LIU Q, LIN B Y, et al. LoraHub: efficient cross-task generalization via dynamic LoRA composition[EB/OL]. [2024-10-15]. https://arxiv.org/pdf/2307.13269.

[9] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks[C]// Proceedings of the 34th International Conference on Neural Information Processing Systems. Red Hook: Curran Associates Inc., 2020: 9459-9474.

[10] DONG Y, WANG S, ZHENG H, et al. Advanced RAG models with graph structures: optimizing complex knowledge reasoning and text generation [C]// Proceedings of the 5th International Symposium on Computer Engineering and Intelligent Communications. Piscataway: IEEE, 2024: 626-630.

[11] GAO Y, XIONG Y, WANG M, et al. Modular RAG: Transforming RAG systems into LEGO-like reconfigurable frameworks [EB/OL]. [2024-10-15]. https://arxiv.org/pdf/2407.21059.

[12] 公诚管理咨询有限公司. 一种基于大模型的检索增强型智能问答方法及系统: 202410949921.6[P]. 2024-08-16.

[13] WANG X, WANG Z, GAO X, et al. Searching for best practices in retrieval-augmented generation [C]// Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. Stroudsburg: ACL, 2024: 17716-17736.

[14] GAO Y, XIONG Y, GAO X, et al. Retrieval-augmented generation for large language models: a survey [EB/OL]. [2024-10-15]. https://arxiv. org/pdf/2312. 10997.

[15] LIANG X, NIU S, LI Z, et al. Empowering large language models to set up a knowledge retrieval indexer via self-learning[EB/OL]. [2024-10-15]. https://arxiv. org/pdf/2405. 16933.

[16] PAN X, VAN OSSENBRUGGEN J, DE BOER V, et al. A RAG approach for generating competency questions in ontology engineering [C]// Proceedings of the 2024 Research Conference on Metadata and Semantics Research, CCIS 2331. Cham: Springer, 2025: 70-81.

[17] ZHOU D, SCHÄRLI N, HOU L, et al. Least-to-most prompting enables complex reasoning in large language models[EB/OL]. [2024-10-15]. https://arxiv. org/pdf/2205. 10625.

[18] MA X, GONG Y, HE P, et al. Query rewriting for retrieval-augmented large language models [C]// Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Stroudsburg: ACL, 2023: 5303-5315.

[19] GAO L, MA X, LIN J, et al. Precise zero-shot dense retrieval without relevance labels[C]// Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Stroudsburg: ACL, 2023: 1762-1777.

[20] ZHENG H S, MISHRA S, CHEN X, et al. Take a step back: evoking reasoning via abstraction in large language models[EB/OL]. [2024-10-15]. https://arxiv. org/pdf/2310. 06117.

[21] CAO H. Recent advances in text embedding: a comprehensive review of top-performing methods on the MTEB benchmark[EB/OL]. [2024-10-15]. https://arxiv. org/pdf/2406. 01607.

[22] LI Z, ZHANG X, ZHANG Y, et al. Towards general text embeddings with multi-stage contrastive learning[EB/OL]. [2024-10-15]. https://arxiv. org/pdf/2308. 03281.

[23] YANG H, LI Z, ZHANG Y, et al. PRCA: fitting black-box large language models for retrieval question answering via pluggable reward-driven contextual adapter[C]// Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Stroudsburg: ACL, 2023: 5364-5375.

[24] LIU N F, LIN K, HEWITT J, et al. Lost in the middle: how language models use long contexts [J]. Transactions of the Association for Computational Linguistics, 2024, 12: 157-173.

[25] LYU Y, LI Z, NIU S, et al. CRUD-RAG: a comprehensive Chinese benchmark for retrieval-augmented generation of large language models [J]. ACM Transactions on Information Systems, 2025, 43 (2): No. 41.

[26] XIA L, XU J, LAN Y, et al. Learning maximal marginal relevance model via directly optimizing diversity evaluation measures [C]// Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval. New York: ACM, 2015: 113-122.

[27] YANG L, HANJALIC A. Supervised reranking for web image search [C]// Proceedings of the 18th ACM International Conference on Multimedia. New York: ACM, 2010: 183-192.

[28] JIANG H, WU Q, LUO X, et al. LongLLMLingua: accelerating and enhancing LLMs in long context scenarios via prompt compression[C]// Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Stroudsburg: ACL, 2024: 1658-1677.

[29] LITMAN R, ANSCHEL O, TSIPER S, et al. SCATTER: selective context attentional scene text recognizer[C]// Proceedings of the 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition. Piscataway: IEEE, 2020: 11959-11969.

[30] CUI J, NING M, LI Z, et al. ChatLaw: open-source legal large language model with integrated external knowledge bases [EB/OL]. [2024-10-15]. https://arxiv. org/pdf/2306. 16092.

[31] SCHICK T, DWIVEDI-YU J, DESSÌ R, et al. Toolformer: language models can teach themselves to use tools[C]// Proceedings of the 37th International Conference on Neural Information Processing Systems. Red Hook: Curran Associates Inc. , 2023: 68539-68551.

[32] OUYANG L, WU J, JIANG X, et al. Training language models to follow instructions with human feedback[C]// Proceedings of the 36th International Conference on Neural Information Processing Systems. Red Hook: Curran Associates Inc. , 2022: 27730-27744.

[33] LIN X V, CHEN X, CHEN M, et al. RA-DIT: retrieval-augmented dual instruction tuning[EB/OL]. [2024-10-15]. https://arxiv. org/pdf/2310. 01352.

[34] SEMNANI S J, YAO V Z, ZHANG H C, et al. WikiChat: stopping the hallucination of large language model chatbots by few-shot grounding on Wikipedia [C]// Findings of the Association for Computational Linguistics: EMNLP 2023. Stroudsburg: ACL, 2023: 2387-2413.

[35] BAEK J, JEONG S, KANG M, et al. Knowledge-augmented language model verification [C]// Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Stroudsburg: ACL, 2023: 1720-1736.

[36] ZHANG J. Graph-Toolformer: to empower LLMs with graph reasoning ability via prompt augmented by chatgpt [EB/OL]. [2024-10-15]. https://arxiv. org/pdf/2304. 11116.

[37] ASAI A, WU Z, WANG Y, et al. Self-RAG: learning to retrieve, generate, and critique through self-reflection[EB/OL]. [2024-10-15]. https://arxiv. org/pdf/2310. 11511.

[38] LUO L, LI Y F, HAFFARI G, et al. Reasoning on graphs: faithful and interpretable large language model reasoning[EB/OL]. [2024-10-15]. https://arxiv. org/pdf/2310. 01061.

[39] CORMACK G V, CLARKE C L A, BUETTCHER S. Reciprocal rank fusion outperforms condorcet and individual rank learning methods [C]// Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval. New York: ACM, 2009: 758-759.